

Linux File Hashing Lab

Generating & Comparing SHA-256 Hash Values to Detect File Tampering

Domain	Tools Used	Difficulty	Status
Cryptography / Linux	sha256sum, cmp, cat	Beginner	Completed

Project Overview

This lab is part of the Google Cybersecurity Certificate curriculum. Hash functions are a cornerstone of data integrity verification in cybersecurity. A hash function produces a fixed-length code (digest) that uniquely represents the contents of a file. Even a single changed character produces a completely different hash value — making hashing an extremely powerful tool for detecting file tampering or corruption.

In this lab, I used Linux command-line tools to generate SHA-256 hash values for two files that appeared visually identical, then compared the hashes to determine whether the files truly contained the same data.

Background: Hash Functions & File Integrity

A hash function takes an input (a file, a string, any data) and produces a fixed-length output called a hash value or digest. The key properties that make hashing useful for security are:

- Deterministic — the same input always produces the same hash
- One-way — you cannot reverse-engineer the original data from the hash
- Avalanche effect — even a tiny change to the input produces a completely different hash
- Collision-resistant — it is computationally infeasible for two different inputs to produce the same hash

SHA-256 (Secure Hash Algorithm 256-bit) is part of the SHA-2 family and is one of the most widely used hashing algorithms in the industry. It produces a 256-bit (64 hexadecimal character) digest and is used in TLS/SSL, digital signatures, blockchain, and file verification tools.

Scenario

Two files — file1.txt and file2.txt — are present in the home directory and appear to contain identical data when displayed with the cat command. The goal is to determine whether these files are truly identical by generating and comparing their SHA-256 hash values.

Task 1: Generate Hashes for Files

Step 1 — List the directory contents

Used `ls` to confirm the files present in the working directory:

```
ls
```

Output: `file1.txt` and `file2.txt` were listed.

Step 2 — Display the contents of both files

Used the `cat` command to view each file:

```
cat file1.txt  
cat file2.txt
```

Output (both files):

```
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

Both files appeared to display identical content. However, visual inspection alone is not sufficient to confirm two files are identical — a single invisible or non-printable character difference would not be apparent to the eye.

Step 3 — Generate the SHA-256 hash of file1.txt

```
sha256sum file1.txt
```

Output:

```
131f95c51cc819465fa1797f6ccacf9d494aaff46fa3eac73ae63ffbfd8267  file1.txt
```

Step 4 — Generate the SHA-256 hash of file2.txt

```
sha256sum file2.txt
```

Output:

```
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
```

Key finding: The two hash values are completely different, confirming that despite appearing visually identical, the files do NOT contain the same data. This demonstrates the avalanche effect of SHA-256.

Task 2: Compare Hashes

In this task the hash values were written to separate files so they could be compared using the `cmp` command, which performs a byte-by-byte comparison and reports exactly where the first difference occurs.

Step 1 — Write hash of file1.txt to file1hash

The `>>` operator appends the output to a new file:

```
sha256sum file1.txt >> file1hash
```

Step 2 — Write hash of file2.txt to file2hash

```
sha256sum file2.txt >> file2hash
```

Step 3 — Display both hash files for visual inspection

```
cat file1hash  
cat file2hash
```

Viewing the hash files confirms the two digests are completely different strings — they do not share the same characters.

Step 4 — Use `cmp` to pinpoint the exact difference

The `cmp` command compares two files byte by byte and reports the position of the first difference:

```
cmp file1hash file2hash
```

Output:

```
file1hash file2hash differ: char 1, line 1
```

The output confirms the files differ starting at character 1, line 1 — meaning the very first character of the two hash values is different. This is consistent with SHA-256's avalanche effect: even a minor difference between the input files produces entirely different hash outputs from the very first character.

Commands Summary

Command	Purpose	Example Used
<code>ls</code>	List directory contents	<code>ls</code>
<code>cat</code>	Display file contents	<code>cat file1.txt</code>
<code>sha256sum</code>	Generate SHA-256 hash of a file	<code>sha256sum file1.txt</code>
<code>>></code>	Append output to a file	<code>sha256sum file1.txt >> file1hash</code>

cmp	Compare two files byte by byte	cmp file1hash file2hash
-----	--------------------------------	-------------------------

Key Takeaways

- Visual inspection of file contents is NOT a reliable method of verifying file integrity — two files can appear identical to the human eye while containing different data.
- SHA-256 generates a unique 256-bit (64 hex character) digest for any file. Any modification to the file — however minor — will produce a completely different hash.
- The sha256sum command is a built-in Linux tool for generating SHA-256 hashes and is widely used in software distribution to verify download integrity.
- The >> operator redirects command output to a file (appending rather than overwriting), which is useful for saving results for later comparison.
- The cmp command provides precise byte-level comparison between files, reporting the exact character and line where the first difference is found.
- Hash comparison is a fundamental technique in digital forensics, malware analysis, and incident response to verify whether files have been modified or tampered with.

Skills Demonstrated

Technical Skills	Conceptual Knowledge
• Linux command-line proficiency • Generating SHA-256 hashes (sha256sum) • Redirecting output to files (>>) • Byte-level file comparison (cmp) • Reading and interpreting hash digests	• Hash function properties (one-way, deterministic) • SHA-256 algorithm and its security properties • Avalanche effect in cryptographic hashing • File integrity verification principles • Practical forensics: detecting file tampering

Conclusion

This lab demonstrated a critical principle in cybersecurity: never trust what you can see alone. Two files that appeared visually identical were proven to contain different data through SHA-256 hashing. This technique is used daily by security analysts — from

verifying downloaded software packages to detecting tampered evidence in digital forensics investigations.